Research 2026-10-21: MCP security, tokens, Supabase OAuth, AI-edit review, invitee data (feeds phase 6, docs/04)

Tags: VERIFIED = first-party page read; SNIPPET = search excerpt only; UNVERIFIED = not confirmed. Not legal advice.

Bottom line

docs/04 is right in direction but outdated in four places: Supabase OAuth is now verified as a public beta; Client ID Metadata Documents (CIMD) are missing; there is no per-client matrix (most clients can send an Authorization header, so path tokens are only needed for the Gemini app); the Supabase OAuth risk that tokens carry full user privileges is not mentioned. Phase 6 order is wrong: verifying clients (6.6) must come BEFORE OAuth (6.5). The path token is acceptable only as a labelled, short-lived, read-only fallback.

MCP status (VERIFIED, modelcontextprotocol.io/specification/2026-07-28/...)

Latest revision 2026-07-28: stateless (Mcp-Session-Id and initialize removed, `server/discover` added), tasks moved to an extension, multi round-trip `input_required`, `tools/list` results carry `ttlMs` and `cacheScope`, Dynamic Client Registration (DCR) deprecated in favour of CIMD, clients must validate `iss` (RFC 9207), HTTP+SSE deprecated. Authorization: the server is an OAuth 2.1 resource server with Protected Resource Metadata (RFC 9728); clients send RFC 8707 `resource`; servers validate audience and never forward tokens; tokens go in `Authorization: Bearer` and MUST NOT be in the query string; 401 with `WWW-Authenticate` carrying `resource_metadata`. Which clients implement 2026-07-28: UNVERIFIED.

Client	Header token	OAuth	Notes
Claude Code	`--header "Authorization: Bearer …"`, `headersHelper`	`/mcp` login, CIMD, DCR, `--client-id`	VERIFIED code.claude.com/docs/en/mcp. A rejected configured header does not fall back to OAuth. Redirect bug v2.1.229 (127.0.0.1 vs localhost), fixed in v2.1.231. No OAuth in `claude -p`.
claude.ai / Desktop / mobile connectors	"Request headers" beta, limited orgs	CIMD or DCR, redirect https://claude.ai/api/mcp/auth_callback	VERIFIED claude.com/docs/connectors/building/authentication. Uses CIMD only if metadata advertises `client_id_metadata_document_supported: true` and `none` auth; else DCR; needs a real 401 with `resource_metadata`; first `authorization_servers` entry only; 10 s / 30 s timeouts; docs say never accept tokens in the URL.
Cursor	`headers` with `${env:VAR}`	static client id or DCR	VERIFIED cursor.com/docs/context/mcp
VS Code	`headers` with `${input:…}`	`oauth.clientId`	VERIFIED
Gemini CLI	`headers`	401 discovery, DCR, `iss` check	VERIFIED geminicli.com/docs/tools/mcp-server/; page says replaced by "Antigravity CLI" for unpaid/Google One users on 2026-06-18
ChatGPT developer mode	not confirmed	OAuth (CIMD/DCR), none, mixed	SNIPPET; plan eligibility conflicts
Gemini app	UNVERIFIED	UNVERIFIED	Google help (support.google.com/gemini/answer/17209137): add a custom app by MCP server URL on web, US, English, personal account, 18+, Keep Activity on; if no DCR enter credentials manually; writes need confirmation. Plan requirement unclear.

Attacks and mitigations

Official page /specification/2026-07-28/basic/security_best_practices (VERIFIED): confused deputy, token passthrough, SSRF, state hijack, OAuth URL injection, mix-up, localhost redirect impersonation, CIMD trust, scope minimization. We are not a third-party proxy, but with Supabase as authorization server our consent page must name the client and redirect URI, have CSRF protection, anti-framing headers and exact redirect matching; enabling CIMD means we fetch client URLs (SSRF). Tool poisoning/rug pull: practitioner write-ups only (SNIPPET: Willison 2025-07-06, Zuplo, Invariant); mitigation = pin tool definitions by hash on the client. Our real risk is OUR documents as the injection channel (lethal trifecta). Missing in docs/04: exfiltration through rendering — Markdown in proposals can carry remote images/links; if the Review inbox auto-loads them, injected text can leak content before a human accepts.

Tokens

GitHub format (VERIFIED github.blog): type prefix + underscore + 32-bit CRC32 checksum in the last 6 characters, base62; prefix alone cuts false positives to ~0.5%. Partner secret-scanning programme needs a public HTTPS verifier endpoint: too heavy for a solo project; use the format and a gitleaks rule. OWASP: credentials must not be in URLs (logs). SHA-256 of a high-entropy token is adequate (judgement). Path token: Supabase edge logs and Vercel logs contain it (retention unconfirmed) -> only read-only, 24 h to 7 d, low daily limit, one per client, revoked on rotation.

Supabase Auth OAuth 2.1 server (VERIFIED supabase.com/docs/guides/auth/oauth-server/..., changelog 38022)

Public beta (GA target Q4 2025 passed; no GA notice found — check live docs). Available on all plans; signed-in users count to MAU. You supply the consent page ("Authorization Path"). PKCE S256 required; DCR off by default; no CIMD documented. Phase 1 has no scopes: tokens carry full user privileges and rely on RLS, so a leaked MCP token is a normal user JWT that works against PostgREST. `aud` changes only via a custom access token hook; RFC 8707 `resource` not mentioned. Documented defences: RESTRICTIVE RLS on `auth.jwt()->>'client_id' IS NOT NULL`, and an `aud` hook that rejects unknown clients. Access tokens 3600 s, refresh rotates; HS256 default (RS256/ES256 recommended).

Human review of AI edits

Sourced: Google Docs suggestions (colour-tracked, nothing changes until accepted, accept one by one or all, owner emailed — support.google.com/docs/answer/6033474); Claude Code manual vs accept-edits mode (code.claude.com/docs/en/security); Claude connectors tool-level Blocked/Always allow; ChatGPT write actions confirm by default (SNIPPET). Cursor, Copilot, Notion AI, Grammarly, Outline pages could not be fetched (omitted). Over-trust research is thin (SNIPPET arXiv 2412.18531): reviewers dismiss issues assuming the bot would flag them; polished output lowers scepticism; usual mitigation = deterministic gate. Audit trails (SNIPPET, vendor blogs): hash chain per row, optional HMAC, signed checkpoints, triggers blocking UPDATE/DELETE, chain head anchored outside the database; the owner is also DB admin so anchoring matters more than the chain.

Invitee data

Could not fetch GDPR Art. 17 text (one-month deadline is from memory: UNVERIFIED). EDPB summary of an Irish case: emailing non-users without consent/legitimate interest was not justified; "compare and forget" accepted (platform contact imports, not exactly our case). Whether GDPR applies to a solo owner's personal use:

UNVERIFIED. Practical minimum: purge or hash invitee email after accept/expiry; per-person export and delete; log retention period; no tokens or IPs in logs.

Ranked changes

1 (S) run client verification first (Claude Code, Cursor, claude.ai, Gemini app), then make `header` the default token kind and `path` the exception. 2 (S) sanitise the Review inbox: no remote images, link targets as text, diffs as plain text. 3 (S) token format `gfk_…` + CRC32 base62 checksum, show last 4 characters, gitleaks rule in CI. 4 (M) docs/04 items 1-2: two token kinds, rotation with overlap, path tokens 24 h by default. 5 (M) per-token tool and document allow-lists; static tool descriptions; expose a hash of the tool list. 6 (M) OAuth stage: enable Supabase OAuth server with DCR, write consent page (client name, exact redirect URI, CSRF, anti-framing), RESTRICTIVE RLS for `client_id IS NOT NULL`, custom access token hook setting/checking `aud` and rejecting unknown clients, Edge Function returns 401 + `resource_metadata` and validates aud/client_id/iss, keep manual tokens as fallback, test with Claude Code and claude.ai. 7 (M) review friction: reason required for high-risk accepts, no bulk accept for them, auto-accept only a narrow class. 8 (M) invitee data purge/export/delete, log retention. 9 (M) hash-chained audit log + weekly head emailed to an external mailbox. 10 (M) anomaly alerts + auto-pause. 11 (L) support spec 2026-07-28 once the clients you use adopt it. Defer: two-person approval, GitHub partner programme. Risk: building the OAuth stage on a beta feature with full-privilege tokens and no RESTRICTIVE RLS would widen exposure beyond today's read-only path tokens.

Sources (fetched 2026-10-21)

modelcontextprotocol.io/specification/2026-07-28/{basic/authorization, changelog, basic/security_best_practices} · claude.com/docs/connectors/{building/authentication, custom/add-unlisted} · code.claude.com/docs/en/{mcp, security} · cursor.com/docs/context/mcp · code.visualstudio.com/docs/agents/reference/mcp-configuration · geminicli.com/docs/tools/mcp-server/ · supabase.com/docs/guides/auth/oauth-server{,/getting-started,/mcp-authentication,/token-security} · supabase.com/changelog/38022-oauth-2-1-server-capabilities-for-supabase-auth · github.blog/engineering/platform-security/behind-githubs-new-authentication-token-formats/ · docs.github.com/en/code-security/secret-scanning/secret-scanning-partnership-program/secret-scanning-partner-program · cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html · support.google.com/docs/answer/6033474 · SNIPPET: simonwillison.net/2025/Jul/06/supabase-mcp-lethal-trifecta/, zuplo.com/blog/remote-mcp-server-rug-pull/, arxiv.org/pdf/2412.18531, edpb.europa.eu (ie_2022-04), appmaster.io/blog/tamper-evident-audit-trails-postgresql